CS5730 Operating Systems Midterm (2022/11/22)

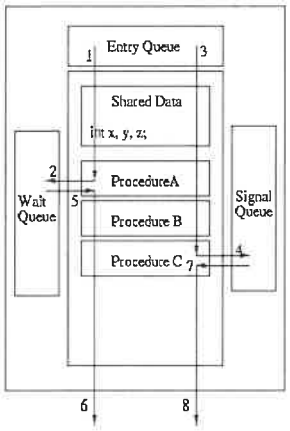
Name:_____
ID#:_____

Essay Questions: Your answers should be as concise and clear as possible

1 (5 pt) Explain why implementing synchronization primitives by disabling interrupts is not appropriate in a single-processor system if the synchronization primitives are to be used in user-level programs.

2 (15 pt) Note that only one process is active within the monitor. Suppose `A` associated with the condition variable `q` is resumed when `B` invokes `q.signal()`. We discussed two possibile solutions in class. One is **signal and wait**, and the flow is illustrated in the following (Hoare semantics). Please modify the Hoare semantics and depict the other solution (i.e., **signal and continue**).

Hoare Semantics



1. ThreadA enters the monitor
2. ThreadA waits for a resource
3. ThreadB enters the monitor
4. ThreadB signals on the resource and enters the signal queue
5. ThreadA re-enters the monitor
6. ThreadA leaves the monitor
7. ThreadB re-enters the monitor
8. ThreadB leaves the monitor
9. Another process can be admitted form the entry queue

3 (20 pt) In class, we have discussed implementing monitor with **signal and wait** via semaphores as follows.

Variables

```
semaphore mutex;  // (initially  = 1)
semaphore next;     // (initially  = 0)
int next-count = 0;
```

Each procedure F will be replaced by

```
wait(mutex);
        ...

 // body of F;

        ...
if (next_count > 0)
      signal(next)
else
      signal(mutex);
```

For each condition variable x,

```
semaphore x_sem; // (initially  = 0)
int x-count = 0;
```

The operation `x.wait` can be implemented as:

```
x-count++;
if (next_count > 0)
      signal(next);
else
      signal(mutex);
wait(x_sem);
x-count--;
```

The operation `x.signal` can be implemented as:

```
if (x-count > 0) {
      next_count++;
      signal(x_sem);
      wait(next);
      next_count--;
}
```

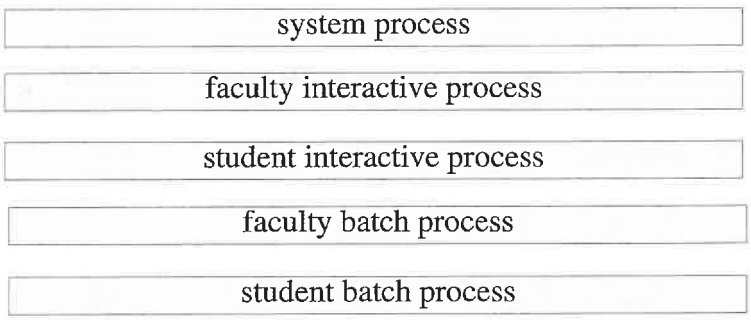
Please modify the above code to implement **signal and continue**.

4 (10 pt) Suppose a architecture use 48-bit addressing with 4K paging on 64-processors. How much memory space a process may have? How many entries a page table may have? How should a page table be stored in memory?

5 Multilevel queue scheduling

5.1 (5 pt) List two possible scheduling among queues, which we discussed in class.

5.2 (5 pt) Modify the following multilevel queue scheduling system such that a new process will automatically enter the appropriate queue after a few runs.



6 Multilevel queue scheduling

6.1 (5 pt) What is **busy-waiting** in the discussion of critical section problem? Explain how **busy-waiting** may degrade the system performance.

6.2 (5 pt) How to implement semaphore without **busy-waiting** using `block` and `wakeup`? You need to write down the definition of data structure `semaphore` and `wait` and `signal` operations.

7 (10 pt) We have discussed the correct solution to the consumer-producer problem, which does not use any synchronization mechanism, as shown below. Please explain why it can only use `BUFFER_SIZE-1` elements of buffer.

Producer:

```
while (true) {
      /* produce an item in next produced */
      while (((in + 1) % BUFFER_SIZE) == out)
            ; /* do nothing */
      buffer[in] = next_produced;
      in = (in + 1) % BUFFER_SIZE;
}
```

Consumer:

```
while (true) {
      while (in == out)
            ; /* do nothing */
      next_consumed = buffer[out];
      out = (out + 1) % BUFFER_SIZE;

      /* consume the item in next consumed */
}
```

8 Consider two processes, $\mathbf{P_1}$ and $\mathbf{P_2}$, where $p_1 = d_1 = 50$, $t_1 = 25$, $p_2 = d_2 = 75$, and $t_2 = 30$.

8.1 (10 pt) Can these two processes be scheduled using rate-monotonic scheduling (RMS)? Illustrate your answer using a Gantt chart such as the ones we did in class.

8.2 (10 pt) Illustrate the scheduling of these two processes using earliest-deadline-first (EDF) scheduling.